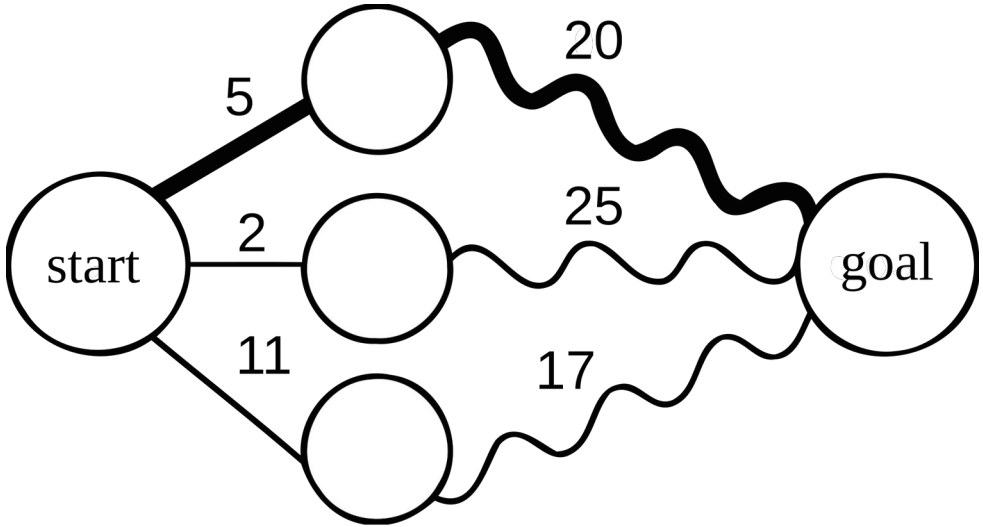
# Lecture 16

## Dynamic Programming II



# Last time

Minimum Change Problem

- USD version has a greedy solution
- General version does not

Dynamic Programming

- Recursive formulation
- Derive recurrence relation
- Bottom-up iterative or top-down memoization algorithm

# Today

Finish Dynamic Programming recipe

- Recover optimal solution from efficient algo

Metal Cutting Problem

# Dynamic Programming Recipe

- Step 1: Devise simple recursive algorithm for the value of the optimal solution
- Step 2: Write recurrence relation for optimal value
- Step 3: Design bottom-up iterative algorithm
- Step 4: Recover optimal solution

# General Minimum Change Problem

**Problem**: You want to give $M$ cents in as few coins as possible using coins worth the following amounts: $C = c_1, c_2, \ldots, c_k$. Assume you can give exact change.

- In general, greedy approach won't work (see prior example)
- Solving via recursive formulation is $O(k^M)$

# Step 1: Recursive Algorithm

```
MinChange(M):
    if M = 0:
           return 0
    opt = []
    for i = 1 to k:
           if c_k ≤ M:
              opt.append(1 + MinChange(M-c_k))
    return min(opt)
```

# Step 2: Recurrence Relation

```
MinChange(M):
    if M = 0:
            return 0
    opt = []
    for i = 1 to k:
            if c_k ≤ M:
                opt.append(1 + MinChange(M-c_k))
    return min(opt)
```

- Can represent optimal solution via a recurrence relation

$$OPT(M) = \begin{cases} 0 & \text{if } M = 0 \\ 1 + min_{c \in C, c \leq M} OPT(M - c) & \text{if } M > 0 \end{cases}$$

# Step 3: Iterative Bottom-Up Algorithm

**Idea**: compute the optimal value of every unique subproblem in order from smallest (base case) to the largest (original problem) using recurrence

- We call this "inductively" computing the optimal value

```
OptChange(M):
    initialize list L of size M to hold optimal values
    L[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
    return L
```

- Running time? `O(Mk)`

# Step 4: Recover the Solution

**Idea**: save the best choice from each subproblem and backtrack

# Step 4: Recover the Solution

**Idea**: save the best choice from each subproblem and backtrack

First, modify `OptChange` to track the choice at each step

```
OptChange(M):
    initialize list L of size M to hold optimal values
    initialize list B of size M to hold optimal choices
    L[0] = 0
    B[0] = 0
    for i = 1 to M:
        opt = []
        for j = 1 to k:
            opt.append(1 + L[max(i – cⱼ, 0)]]
            L[i] = min(opt)
            B[i] = which min(opt)
    return L
```

returns index of min

# Step 4: Recover the Solution

**Idea**: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

# Step 4: Recover the Solution

**Idea**: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

- Running time?

# Step 4: Recover the Solution

**Idea**: save the best choice from each subproblem and backtrack

Then, find out which choices were made

```
getSolution(M):
    given lists L, B
    A = {}
    i = M
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - c_j
    return A
```

- Running time? `O(M)`

# Recap of DP Process

Recursive algorithm -> find recurrence -> write iterative algorithm

Three ways of expressive value of optimal solutions to subproblems

- `MinChange(M)`: recursive algorithm
- `OPT(M)`: recurrence relation
- `L`: array holding optimal value at each distinct subproblem

# Metal Cutting Problem

Version of a classic problem in operations research

Input:

- Steel rod of length `n`, which can be cut into integer lengths and sold for price $p_i$ for length `i`

Goal:

- Subdivide steel rod to maximize sell price

# Metal Cutting Problem

Version of a classic problem in operations research

Input:

- Steel rod of length `n`, which can be cut into integer lengths and sold for price $p_i$ for length `i`

Goal:

- Subdivide steel rod to maximize sell price

Why won't greedy work here?

# Metal Cutting Problem

Version of a classic problem in operations research

Input:

- Steel rod of length `n`, which can be cut into integer lengths and sold for price $p_i$ for length `i`

Goal:

- Subdivide steel rod to maximize sell price

Why won't greedy work here?

- Local choices cannot capture global constraints

# Recursive Algorithm

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```



# Recursive Algorithm

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

base case

consider all possible next cuts to make

# Recursive Algorithm

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

base case

Running time: $O(2^n)$

consider all possible next cuts to make

# Recurrence Relation

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

$$OPT(m) = \begin{cases} 0 & \text{if } m = 0 \\ max_{1 \le i \le m} \{ p_i + OPT(m-i) \} & \text{if } m > 0 \end{cases}$$

# Recurrence Relation

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

$$OPT(m) = \begin{cases} 0 & \text{if } m = 0 \\ max_{1 \le i \le m} \{ p_i + OPT(m-i) \} & \text{if } m > 0 \end{cases}$$

- The recurrence provides all info needed to design an iterative algorithm
- Start from 0 then iterate upward

# Iterative Algorithm

```
CutMetalItr(n):
    L = []
    L[0] = 0
    for i = 1 to n:
        v = 0
        for j = 1 to i:
            v = max(v, p[j] + L[i-j])
        L[i] = v
    return L
```

# Iterative Algorithm

```
CutMetalItr(n):
    L = []
    L[0] = 0
    for i = 1 to n:
        v = 0
        for j = 1 to i:
            v = max(v, p[j] + L[i-j])
        L[i] = v
    return L
```

Running time: $\Theta(n^2)$

# Retrieve Optimal Solution

```
CutMetalItr(n):
    L = []
    B = []
    L[0] = 0
    B[0] = 0
    for i = 1 to n:
        v = 0
        idx = 0
        for j = 1 to i:
            if p[j] + L[i-j] > v:
                v = p[j] + L[i-j]
                idx = j
        L[i] = v
        B[i] = idx
    return L, B
```

# Retrieve Optimal Solution

```
CutMetalItr(n):
    L = []
    B = []
    L[0] = 0
    B[0] = 0
    for i = 1 to n:
        v = 0
        idx = 0
        for j = 1 to i:
            if p[j] + L[i-j] > v:
                v = p[j] + L[i-j]
                idx = j
        L[i] = v
        B[i] = idx
    return L, B
```

```
getSolution(M):
    given lists L, B
    A = {}
    i = n
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - j
    return A
```

# Next Time

- Pseudo-polynomial complexity
- Knapsack Problem